■ Bellman–Ford Algorithm (Routing)

■ Formula

For each edge (u → v) with cost/weight w(u,v):

D(v) = min(D(v), D(u) + w(u,v))

• D(v) = shortest distance to node v
• D(u) = shortest distance to node u
• w(u,v) = weight (cost/delay/hop) of edge u → v

■ Repeat this relaxation for all edges, up to (V – 1) times, where V = number of vertices.

■ Steps

1. Initialize distances:
   • D(source) = 0
   • D(all others) = ∞

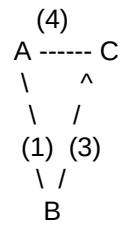
2. Repeat (V – 1) times:
   • For each edge (u, v): apply formula.

3. After V – 1 iterations → shortest distances found.

4. Optional: Run one more iteration to check negative cycle.
   • If any distance improves → negative cycle exists.

■ Example (Step by Step)

Graph:



Edges:
• A → B = 1
• B → C = 3
• A → C = 4
Step 1: Initialization
• D(A) = 0
• D(B) = ∞
• D(C) = ∞
Step 2: Iteration 1 (relax all edges)
• Edge A → B: D(B) = min(∞, 0+1) = 1
• Edge B → C: D(C) = min(∞, 1+3) = 4
• Edge A → C: D(C) = min(4, 0+4) = 4 (no change)

Now:
• D(A)=0, D(B)=1, D(C)=4
Step 3: Iteration 2 (again relax edges)
• A → B: already 1, no change.
• B → C: D(C) = min(4, 1+3) = 4 (no change).
• A → C: already 4.

No changes → algorithm stops early.

■ Final shortest paths:
• A → B = 1
• A → C = 4 (either direct or via B)
■ General Routing Form (used in Distance Vector)

For router x to reach destination y:

Dx(y) = min_{v in N(x)} (c(x,v) + Dv(y))

Where:
• Dx(y) = cost from router x → y
• c(x,v) = cost of link (x,v)
• Dv(y) = cost from neighbor v → y
• N(x) = neighbors of x

■ This is exactly what RIP (Routing Information Protocol) does.